

TP1

Computación Gráfica Aplicada y Sistemas Generativos Cátedra Causa | Facultad de Artes | UNLP | 2026

TP1 Bitácora de proceso con IA 2026

Comisión Matias / Lisandro

Estudiantes:

- Valcarcel Joaquín 122933/6
- Paulina Tassara 122915/4
- Francisco Torres 122923/4
- Guida Matilda 122722/7

Qué es: un registro documentado del proceso de trabajo con IA, que incluye los prompts clave, los resultados visuales obtenidos, las correcciones realizadas, y los aprendizajes. No es un log de todas las conversaciones — es una selección curada de los momentos significativos del desarrollo.

TECNOLOGÍAS UTILIZADAS:

Entorno / Plataforma de trabajo:

Claude Code (extensión de VSCode)

Modelo de IA utilizado:

Claude Sonnet 4.6

Entrada #1 — 22/05/2026 — Estructura base: franjas animadas inspiradas en Bridget Riley

1. Objetivo

Crear desde cero una obra generativa en HTML + p5.js que replique el estilo Op Art de Bridget Riley: franjas verticales deformadas por funciones seno superpuestas, con animación continua y paleta de colores neutros.

2. Prompt

"Quiero crear una obra de arte generativa en HTML con p5.js inspirada en las ondas de Bridget

Riley. Que tenga franjas animadas que se muevan con ondas sinusoidales, paletas de colores suaves y terrosos, y que la interacción básica sea con teclado y mouse."

3. Resultado visual

[Captura del canvas: 560×920px, 32 franjas verticales de colores suaves — verde salvia, azul pálido, beige, gris, naranja — oscilando suavemente con dos funciones seno superpuestas.]

4. Evaluación y corrección

El resultado se acercó al estilo de Riley desde el primer intento. p5.js generó las franjas usando beginShape/vertex, que permite construir polígonos con bordes irregulares. Las dos funciones seno superpuestas (una de baja frecuencia y alta amplitud, otra de alta frecuencia y baja amplitud) generaron la complejidad orgánica característica de la artista. La paleta era coherente con la estética: colores cálidos y desaturados sobre fondo beige. No fue necesario corregir nada estructural en esta etapa.

5. Aprendizaje

Para generar franjas onduladas en p5.js la técnica más eficiente es trazar el borde izquierdo de arriba a abajo y el borde derecho de abajo a arriba, cerrando el polígono con endShape(CLOSE). Superponer dos funciones seno con frecuencias y amplitudes distintas produce una ondulación más orgánica y menos mecánica que usar una sola.

Entrada #2 — 22/05/2026 — Sistema de paletas con transición suave y bug en la tecla C

1. Objetivo

Implementar tres paletas de color y una tecla C que las cicle con transición animada mediante interpolación lineal (lerp), para que el cambio no sea abrupto sino gradual.

2. Prompt

"Agrega un sistema de 3 paletas de color. Que al presionar C cambie a la siguiente paleta con una transición suave usando lerp. Cada paleta tiene colores distintos para las franjas y el fondo."

3. Resultado visual

[Captura mostrando la transición entre paletas: las franjas pasan gradualmente de verde/azul/naranja a una combinación diferente en ~1 segundo.]

4. Evaluación y corrección

La transición animaba correctamente durante ~1 segundo, pero al completarse los colores volvían a los anteriores. El bug estaba en la función `getStripe()`: cuando `lerpT` alcanzaba 1 (transición completa), la función devolvía `currentColors` — que era el snapshot del estado previo a la transición — en lugar de `targetColors` (la paleta nueva). La animación interpolaba bien, pero al terminar "revertía" visualmente. La corrección fue hacer un commit al finalizar la transición: cuando `lerpT >= 1`, asignar `currentColors = targetColors`. Así los colores nuevos quedan confirmados como estado actual.

5. Aprendizaje

En sistemas de interpolación de estados, cuando la transición completa hay que "confirmar" el estado nuevo sobrescribiendo el anterior. Si no se hace este paso, el resultado visible siempre vuelve al snapshot inicial porque la condición `lerpT >= 1` hace referencia al estado viejo.

Entrada #3 — 22/05/2026 — Click para acelerar y espacio para ralentizar: el bug del estado

1. Objetivo

Agregar dos modos de velocidad controlados por el usuario: mantener el mouse apretado para que la animación acelere progresivamente, y presionar la barra espaciadora para que se ralentice mientras se la sostiene.

2. Prompt

"Quiero que al hacer click y mantener el mouse, las ondas se aceleren progresivamente hasta una velocidad máxima. Y que mientras se mantiene apretada la barra espaciadora el movimiento se ralentice notablemente."

3. Resultado visual

[Captura comparativa: ondas en velocidad normal / ondas en velocidad acelerada con el mouse apretado.]

4. Evaluación y corrección

El click-hold funcionó correctamente. El espacio no funcionaba en absoluto. El problema era de lógica de estado: en `keyPressed` se asignaba `targetSpeed = 0.1`, pero en cada frame del loop `draw()` se evaluaba: si `holding == false` → `targetSpeed = 0.4`, lo que pisaba inmediatamente el

valor que `keyPressed` había asignado. La corrección fue agregar una variable booleana `slowed` persistente: se activa en `keyPressed` (espacio), se desactiva en `keyReleased` (soltar espacio), y en `draw()` se evalúa como tercer estado posible. También se agregó `return false` en `keyPressed` para evitar que el browser interprete la barra espaciadora como scroll de página.

5. Aprendizaje

Las teclas que deben tener efecto mientras se mantienen apretadas no pueden manejarse con asignaciones directas en `keyPressed`: el loop de animación las sobrescribe en el siguiente frame. La solución correcta es usar una variable de estado booleana que persiste entre frames y se desactiva en `keyReleased`.

Entrada #4 — 22/05/2026 — Nuevas interacciones: inversión de dirección y pausa

1. Objetivo

Explorar qué interacciones adicionales enriquecen la obra sin romper su estética. Se buscó agregar una forma de invertir el flujo de las ondas y una forma de congelar la animación en un fotograma específico.

2. Prompt

"¿Se te ocurre alguna interactividad más que puedas sumarle a la obra?"

(La IA propuso y el usuario aceptó:)

- Tecla I: invertir la dirección del movimiento de las ondas
- Tecla F: congelar / pausar la animación

3. Resultado visual

[Captura de la obra pausada en un fotograma: las ondas detenidas en una configuración visual estática, como si fuera una serigrafía.]

4. Evaluación y corrección

La inversión de dirección (I) es visualmente inmediata y llamativa: las ondas que fluían hacia abajo pasan a fluir hacia arriba, generando una sensación de "rebobinado". La pausa (F) se implementó con `p5.noLoop()` y `p5.loop()` en lugar de un `return` condicional dentro de `draw()`. Esto es más eficiente porque detiene completamente el loop de renderizado sin consumir CPU. Ambas funciones se aceptaron sin corrección.

5. Aprendizaje

p5.js tiene noLoop() y loop() como mecanismo nativo de pausa. Es preferible a poner un condicional al inicio de draw() porque libera el hilo de renderizado completamente. La tecla I que invierte el signo del offset abre además posibilidades compositivas: la obra puede "tocarse" en dos direcciones.

Entrada #5 — 22/05/2026 — Ajuste de la curvatura: de dinámica a fija

1. Objetivo

Eliminar el efecto del mouse sobre la amplitud de las ondas (que había sido agregado como feature extra) y fijar la curvatura en el valor estético más adecuado para la obra: el equivalente a tener el mouse pegado al extremo izquierdo del canvas.

2. Prompts

Primero:

"No quiero que al mover el mouse se modifique la curvatura, quiero que se mantenga igual siempre."

Luego, al ver el resultado con amplitud 1.0 (valor por defecto):

"¿Podés hacer que la curvatura de las líneas sea como si el mouse estuviese siempre a la izquierda de la pantalla?"

3. Resultado visual

[Captura del resultado final: franjas con curvatura sutil y contenida, ondulación elegante sin exageración, sobre fondo beige.]

4. Evaluación y corrección

El sistema dinámico usaba p.map(mouseX, 0, W, 0.35, 2.0) para calcular un multiplicador de amplitud. Cuando se eliminó el sistema, la amplitud quedó en 1.0 (el valor por defecto). La curvatura resultante era visualmente diferente a lo que el usuario quería. Al pedir "como si el mouse estuviese a la izquierda", se tradujo a: el valor 0.35 del extremo izquierdo del mapeo anterior. Se reemplazaron las constantes AMP de 38 a 13 ($38 \times 0.35 \approx 13$) y la onda secundaria de 12 a 4 ($12 \times 0.35 \approx 4$). No fue necesario conservar ningún sistema dinámico.

5. Aprendizaje

Cuando un parámetro que fue dinámico termina siendo siempre el mismo valor, lo más limpio es eliminar el sistema y hardcodear el valor directamente en la constante. Además, para comunicarle a la IA un valor estético difícil de describir con palabras ("quiero menos curvatura"), es útil referirse a un estado anterior que sí funcionaba ("como cuando el mouse estaba a la izquierda") — la IA puede traducir esa referencia al valor numérico exacto.

Entrada #6 — 10/06/2026 — Secuencia exacta de 35 franjas: recreando "Red Dominance" al pie de la letra

1. Objetivo

Corregir el orden y las proporciones de las franjas para que coincidan exactamente con la obra original "Red Dominance" (1977) de Bridget Riley. En el estado anterior los colores tenían el orden incorrecto y todos los anchos eran uniformes, cuando en el original las franjas crema y gris son notablemente más anchas que las de color.

2. Prompt

"de esta version, siento que el orden de los colores esta mal (las lineas mas anchas deberia ser color crema del fondo o gris), ademas de eso, las lineas mas finas no deben llegar hasta el final de la pantalla, solo 1 que toque y listo, que se vea el fondo crema corrige eso. Orden de colores: verde, azul, crema (fondo), azul, verde, crema (fondo), verde, azul, gris, naranja salmon, verde, crema (fondo), verde, naranja salmon, crema (fondo), naranja salmon, verde, gris, verde, azul, crema (fondo), azul, verde, crema (fondo), verde, azul, gris, naranja salmon, verde, crema (fondo), verde, naranja salmon, crema (fondo), naranja salmon, verde. FIN ese es el patron al pie de la letra"

3. Resultado visual

[Captura del canvas: 35 franjas con anchos variables. Las crema/gris miden 18-21px y las de color 11px. Las ondas de cada franja están desincronizadas, generando la ilusión de movimiento diagonal característica de Riley.]

4. Evaluación y corrección

La IA implementó un array STRIPE_SEQ de 35 pares [color, ancho] con cinco constantes de ancho: WV=11 (verde), WA=11 (azul), WS=11 (salmón), WC=18 (crema fondo), WG=21 (gris). A partir de ese array construyó el array BOUNDS, donde cada límite entre franjas acumula un phase offset que se incrementa 0.40 por franja (PHASE_STEP=0.40). Esta acumulación de fase es lo que genera la desincronización visual: cada franja "ondula" con un retraso diferente respecto a las demás, creando la apariencia de movimiento diagonal. El resultado coincidió

visualmente con el original sin correcciones adicionales.

5. Aprendizaje

Para recrear fielmente una obra existente, el prompt más efectivo fue dictar la secuencia literal de 35 elementos en lugar de intentar describir el patrón con palabras. La acumulación de phase offset (± 0.40 por franja) es el mecanismo clave de la obra: sin él, todas las franjas ondulan en fase y el resultado parece una cortina, no una composición Op Art. Ese único parámetro define el carácter visual de la pieza.

Entrada #7 — 10/06/2026 — AMP=17: las franjas finas se adelgazan al curvarse sobre las anchas

1. Objetivo

Amplificar el efecto visual de que las franjas de color se "adelgazan" cuando su curvatura las lleva a ocupar el espacio de las franjas más anchas (crema y gris), reforzando la tensión óptica que es el núcleo estético de Riley.

2. Prompt

"cuando los conjuntos de líneas chicas se curban y ocupan espacio de las líneas mas grandes (gris y crema) debería de esa línea verse mas fina, quizás incrementando la variable de curvatura"

3. Resultado visual

[Comparación antes/después: con AMP=13 las franjas mantienen grosor relativamente uniforme a lo largo del canvas. Con AMP=17 las franjas finas casi desaparecen visualmente en los picos de curvatura donde se meten en el área de las franjas anchas.]

4. Evaluación y corrección

El parámetro AMP controla la amplitud total de la onda sinusoidal que desplaza horizontalmente los bordes de cada franja. Al aumentar AMP de 13 a 17, el desplazamiento horizontal máximo crece, lo que hace que los bordes de las franjas finas se acerquen más entre sí en los picos de curvatura. Como esas franjas ya tienen solo 11px de ancho natural, al comprimirse visualmente tienden a desaparecer. El efecto fue exactamente el buscado sin necesidad de corrección.

5. Aprendizaje

AMP no es solo una variable estética de "qué tanto ondula" — define cuánto puede invadir cada franja el espacio de su vecina. A mayor AMP, mayor tensión perceptual entre las anchas y las delgadas. En Riley ese adelgazamiento es central: es lo que genera la vibración óptica que hace que el ojo perciba movimiento donde no lo hay. Subir AMP de 13 a 17 fue un ajuste fino que capturó esa cualidad.

Entrada #8 — 10/06/2026 — Control por micrófono: integración de Web Audio API

1. Objetivo

Reemplazar los controles de teclado como sistema principal de interacción por gestos sonoros captados por micrófono, usando la Web Audio API del navegador directamente (sin librerías externas), para que la obra pueda "tocarse" de forma performativa en vivo.

2. Prompt

"Igualmente todas las animaciones de la obra la tendremos que hacer con sonidos. (Incremento de velocidad: Sonido grave, decrecimiento de velocidad: Sonido agudo, sin sonido: pieza intacta solo moviéndose, chasquido: cambio de dirección de movimiento, SHHH: reseteo a valores iniciales, aplauso: cambio de color, W: cambio de anchura en líneas (este nose que sonido ponerle para que no se cruce con otra interacción))"

3. Resultado visual

[Captura mostrando el botón "  Activar micrófono" en la barra inferior del canvas, y el indicador "● activo" cuando el micrófono está conectado y funcionando.]

4. Evaluación y corrección

La IA implementó un sistema con AudioContext + AnalyserNode que opera en paralelo al loop de animación de p5.js. En cada frame de draw() se llama a audioTick() que lee la FFT (getByteFrequencyData) y la señal temporal (getFloatTimeDomainData). Se dividió el espectro en tres bandas: bass (0–300 Hz), mid (300–2500 Hz), high (2500+ Hz). Los sonidos graves y agudos funcionaron correctamente desde el primer intento con detección por dominancia de banda. Los transitorios (chasquido, aplauso) y el SHHH necesitaron corrección en iteraciones posteriores por requerir técnicas de detección más específicas.

5. Aprendizaje

getByteFrequencyData() entrega la energía por bin de FFT en escala 0–255, ideal para

comparar bandas de frecuencia. `getFloatTimeDomainData()` entrega la señal temporal en rango -1 a 1, ideal para calcular RMS (raíz del promedio de los cuadrados). Ambos métodos operan sobre el mismo `AnalyserNode` sin conflicto. El parámetro `smoothingTimeConstant` afecta directamente la velocidad de respuesta: un valor alto (>0.7) hace que los cambios de espectro sean lentos, lo que perjudica la detección de onsets percusivos.

Entrada #9 — 10/06/2026 — Bug: flujo espectral, SHHH fricativo y separación de sonidos

1. Objetivo

Corregir tres bugs de detección de audio que se manifestaron en la prueba del sistema: (1) el chasquido y el aplauso se confundían y ambos cambiaban la dirección, (2) el sonido SHHH no activaba el reset, (3) el zumbido vocal disparaba el aumento de velocidad como si fuera un sonido grave.

2. Prompt

"el chasquido y el aplauso suenan similar por lo que haciendo eso cambia de dirección. el SHH no funciona. agudo y grave si. Zumbido actua como si fuese un sonido grave e incrementa la velocidad"

3. Resultado visual

[Captura del overlay de debug (tecla D) mostrando en tiempo real sobre el canvas: rms, flux, bass, mid, high, shush, conteo de onsets activos y cooldowns.]

4. Evaluación y corrección

Bug 1 — Onset detector: el sistema original usaba la diferencia de RMS entre frames consecutivos ($\text{rms} - \text{prevRms}$). Este detector es demasiado insensible: un aplauso genera varias palmadas y la primera levanta el nivel base de RMS, haciendo que las siguientes no superen el umbral. Se reemplazó con flujo espectral: suma de los incrementos positivos en los 1024 bins de FFT entre frames. Este método produce un pico aislado y nítido por cada palmada porque mide la aparición de nuevo contenido espectral, no el nivel absoluto. La máquina de estados también se rediseñó: 1 onset + 0.5s de silencio → cambio de dirección; 2 onsets en $<0.33\text{s}$ → doble-chasquido (widen); 4+ onsets acumulados → cambio de color.

Bug 2 — SHHH: el criterio original era "espectro plano" (todas las bandas similares entre sí). Esto es físicamente incorrecto: el sonido "sh" es una fricativa alveopalatal voiceless cuya energía se concentra en 3000–8000 Hz, con muy poca energía en graves y medios. No es plano, es dominante en altas frecuencias. Se creó una banda específica `shushBand` (promedio

de energía en 3000–8000 Hz) con condición: $\text{shushBand} > 0.07 \ \&\& \ \text{shushBand} > \text{bass} * 1.8 \ \&\& \ \text{rms} > 0.018$, sostenido por más de 40 frames.

Bug 3 — Zumbido: la voz humana al tararear tiene el fundamental en ~200–400 Hz (banda grave) y los armónicos siguientes en medio. Por eso el detector de graves lo capturaba antes de que el detector de medio pudiera actuar. La solución fue eliminar el zumbido como trigger (reemplazado por doble-chasquido) y aumentar el requisito de dominancia del grave: $\text{bass} > \text{mid} * 1.4$.

5. Aprendizaje

Cada tipo de sonido tiene una firma espectral física específica que debe guiar el diseño del detector. Usar "espectro plano" para SHHH era conceptualmente incorrecto porque ignora que el SHHH es una fricativa, no un ruido blanco. El flujo espectral es el método estándar en análisis de audio para detección de onsets percusivos: no mide cuánto ruido hay, sino cuándo aparece ruido nuevo. Esa distinción es crítica para diferenciar el primer chasquido de un aplauso del resto de la secuencia.

Entrada #10 — 10/06/2026 — Nuevo trigger de color: sonido agudo repentino tipo "AAAA"

1. Objetivo

Reemplazar el aplauso como trigger de cambio de color por un sonido agudo de golpe (como "AAA!" o un grito breve agudo), buscando un gesto sonoro que sea más distinguible del chasquido simple y que tenga mayor expresividad performativa.

2. Prompt

"que el cambio de color sea una frecuencia alta de golpe, ejemplo: AAAA."

3. Resultado visual

[Captura del debug overlay en el momento de hacer "AAA!": el valor high supera 0.17, bass está bajo (< 0.08), y se cumple la condición $\text{high} > \text{bass} * 2.2$. El cambio de paleta comienza con la transición lerp.]

4. Evaluación y corrección

La detección combina dos condiciones simultáneas: (1) pico de flujo espectral ($\text{isOnset} = \text{true}$, hay un evento transitorio), (2) la banda de altas frecuencias domina claramente sobre las graves ($\text{high} > \text{bass} * 2.2$). Esta combinación es específica de un sonido agudo repentino: las

palmas tienen energía broadband que incluye graves y no cumplen la condición, mientras que un "AAA!" agudo concentra su energía en altas frecuencias. Se agregó un colorCooldown de 45 frames (~0.75s) para evitar que la condición "agudo sostenido → bajar velocidad" se dispare inmediatamente después del cambio de color, ya que ambas condiciones comparten la banda de altas frecuencias.

5. Aprendizaje

Para distinguir "agudo de golpe" de "agudo sostenido" —que ya tenía asignado el efecto de bajar la velocidad— la clave no es la frecuencia sino la combinación onset + frecuencia. Un onset de alta frecuencia es un evento cualitativamente diferente a una frecuencia alta sostenida, aunque usen la misma banda de análisis espectral. El mecanismo de cooldown post-acción es una herramienta de diseño útil cuando dos condiciones de activación comparten el mismo espacio de señal y podrían solaparse temporalmente.